

SC1008 C/C++ Programming
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1	C Basics: Your First Conversation with the Machine	1
1.1	What Is a C Program?	1
1.2	Types and Variables: Boxes in Memory	2
1.3	Input and Output	2
1.4	Control Flow: Branching and Loops	3
1.4.1	Branching	3
1.4.2	Loops	3
1.5	Functions and Scope	4
1.6	Operators, Expressions, and Precedence	4
1.7	Worked Example: Grading Program	5
1.8	Chapter Notes	5
1.9	Exercises	5
2	Pointers and Memory: Addresses as Data	6
2.1	Addresses and the & Operator	6
2.2	The Stack: Frames on Call	7
2.3	Pointer Arithmetic: Typed Steps	7
2.4	Dangerous Pointers	8
2.5	Const Pointers and Pointer-to-Const	8
2.6	Arrays, Pointers, and Function Parameters	8
2.7	Worked Example: Pass-by-Reference via Pointers	9
2.8	Pointer Safety Checklist	9
2.9	Chapter Notes	9

2.10 Exercises	10
3 Arrays, Strings, and Structs: Building Bigger Boxes	11
3.1 Arrays: Contiguous Homogeneous Storage	11
3.2 C Strings: Char Arrays with a Sentinel	12
3.3 Structs: Heterogeneous Records	12
3.4 Arrays vs. Pointers — Decay	13
3.5 Searching and Sorting Arrays	13
3.6 Nested Structs and Typedef	13
3.7 Common Pitfalls and Safe String Handling	14
3.8 Chapter Notes	14
3.9 Exercises	14
4 Dynamic Memory: The Heap and Its Discipline	16
4.1 Stack vs. Heap	16
4.2 malloc, calloc, realloc, free	17
4.3 Common Heap Bugs	17
4.4 Valgrind and AddressSanitizer	18
4.5 A Resizable Array in C	18
4.6 Deep Copy vs. Shallow Copy	18
4.7 Worked Example: Dynamic String Builder	19
4.8 Chapter Notes	19
4.9 Exercises	19
5 C++ Classes and Object-Oriented Basics	21
5.1 From Struct to Class	21
5.2 Constructors, Destructors, Initialiser Lists	22
5.3 Encapsulation and Invariants	23
5.4 Const Correctness and References	23
5.5 Operator Overloading	23
5.6 Static Members and Friends	24

5.7	Member Functions vs. Non-Members	24
5.8	Chapter Notes	25
5.9	Exercises	25
6	Inheritance, Polymorphism, and Templates	26
6.1	Inheritance: Is-A	26
6.2	Polymorphism and Virtual Dispatch	27
6.3	Templates: Compile-Time Stamping	27
6.4	Static vs. Dynamic Polymorphism	28
6.5	Access Control and Protected Members	28
6.6	Abstract Classes and Interfaces	28
6.7	Template Specialisation and Non-Type Parameters	29
6.8	Chapter Notes	29
6.9	Exercises	30
7	The Standard Template Library: Vectors, Maps, and Iterators	31
7.1	Sequence Containers: vector and string	31
7.2	Associative Containers: map, set, unordered_map	32
7.3	Iterators: Generalised Pointers	32
7.4	Choosing a Container	33
7.5	STL Algorithms in Depth	33
7.6	Complexity Guarantees and Big-O of Containers	34
7.7	Chapter Notes	34
7.8	Exercises	34
8	Modern C++: RAII, Smart Pointers, and Move Semantics	35
8.1	RAII: Resource Acquisition Is Initialisation	35
8.2	Smart Pointers: Explicit Ownership	36
8.3	Move Semantics: Stealing Instead of Copying	36
8.4	Rule of Zero / Three / Five	37
8.5	Exception Safety and RAII in Practice	37

8.6	Perfect Forwarding and Modern Idioms	38
8.7	Chapter Notes	38
8.8	Exercises	38

Chapter 1

C Basics: Your First Conversation with the Machine

“C is not a high-level language to do low-level things — it is a low-level language with high-level syntax.” Before objects, templates, or smart pointers, we must speak the machine’s own dialect. This chapter builds it from zero.

From zero: no prior programming assumed. We start with what a program *is* (instructions acting on memory), then build types, variables, I/O, and control from first principles with pictures before formalism. Later chapters depend only on what is built here.

Learning Outcomes

After this chapter you will be able to:

- (L1) describe the compile–run pipeline and the role of `main()`;
- (L2) declare variables with correct C types and explain their memory cost;
- (L3) read and write data with `printf/scanf`;
- (L4) write branching (`if/switch`) and loops (`for/while`);
- (L5) trace execution and predict output of short C programs.

1.1 What Is a C Program?

A computer is a *stored-program* machine: a long array of memory cells, each holding a byte (8 bits), plus a CPU that fetches, decodes and executes instructions one by one. A C program is a recipe written in text that we *compile* (translate) into those machine instructions.

Every C program starts at `main()`:

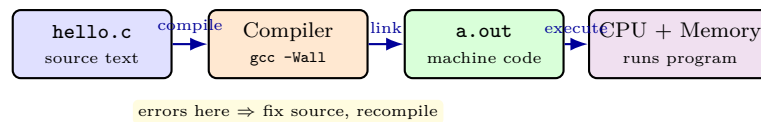


Figure 1.1: The C pipeline: source $\rightarrow$ compiler $\rightarrow$ executable $\rightarrow$ running process. Errors at compile time are cheap; errors at run time are bugs.

```

1 #include <stdio.h>           // header: declarations for printf/scanf
2 int main(void) {             // entry point, returns exit status
3     printf("Hello, NTU!\n");
4     return 0;                 // 0 = success
5 }
  
```

`#include` pastes in declarations; `int main` is the door the operating system knocks on.

1.2 Types and Variables: Boxes in Memory

A *variable* is a named box occupying contiguous bytes in memory. Its *type* says how many bytes and how to interpret the bits.

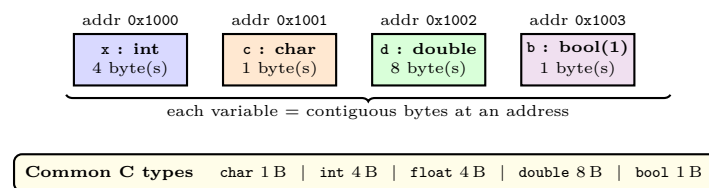


Figure 1.2: Variables as typed byte-boxes in memory. The type dictates size and interpretation; the address is where the box lives.

```

1 int    age = 19;              // 4 bytes, signed -2^31 .. 2^31-1
2 char   grade = 'A';          // 1 byte, single quotes
3 double gpa = 4.75;           // 8 bytes, floating point
4 _Bool  done = 0;              // 0 false, nonzero true (#include <stdbool.h> gives bool)
5 const int MAX = 100;          // const: cannot be reassigned
  
```

C is *statically typed*: the type is fixed at declaration. Initialise always — an uninitialised `int` holds garbage (whatever bits were there before).

Integer promotion and truncation matter: assigning 300 to a `char` keeps only the low 8 bits. Use `sizeof(int)` to query size portably.

1.3 Input and Output

`printf` formats output; `scanf` parses input. Both use *format specifiers* that must match the variable type.

```

1 #include <stdio.h>
2 int main(void){
3     int age; double gpa;
4     printf("Enter age and GPA: ");
  
```

```

5   if (scanf("%d %lf", &age, &gpa) != 2){ // & = address-of (Ch.2)
6       printf("Bad input!\n"); return 1;
7   }
8   printf("Age %d, GPA %.2f\n", age, gpa); // %.2f = 2 decimal places
9   return 0;
10  }

```

Common specifiers: %d int, %c char, %f float, %lf double, %s string (char array). Always check `scanf`'s return value: it counts successful conversions.

Buffering note: `printf` may not appear until newline `\n` or `fflush(stdout)`.

1.4 Control Flow: Branching and Loops

Programs need decisions and repetition. C provides both with minimal syntax.

1.4.1 Branching

```

1   int score = 82;
2   if (score >= 80)    printf("A\n");
3   else if (score >= 70) printf("B\n");
4   else                printf("C\n");
5
6   char op = '+';
7   switch(op){         // switch: jump table on integer/char
8       case '+': printf("add\n"); break;
9       case '-': printf("sub\n"); break;
10      default:  printf("other\n");
11  }

```

`break` exits the `switch`; without it execution *falls through*. Braces `{}` group multiple statements.

1.4.2 Loops

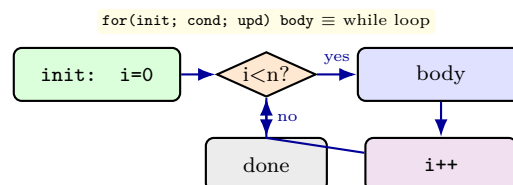


Figure 1.3: Flow of a for loop: initialise $\rightarrow$ test $\rightarrow$ body $\rightarrow$ update $\rightarrow$ test ... until the condition fails.

```

1   for (int i = 0; i < 5; i++)    printf("%d ", i); // 0 1 2 3 4
2
3   int k = 5;
4   while (k > 0){ printf("%d ", k); k--; }          // 5 4 3 2 1
5
6   int n;
7   do { printf("Enter positive n: "); scanf("%d",&n); } while(n <= 0);

```


for is for counted iteration; **while** for condition-driven; **do-while** runs at least once. **break** exits the innermost loop; **continue** skips to next iteration.

1.5 Functions and Scope

A *function* is a named sub-recipe with parameters and a return value. It enables decomposition and reuse.

```

1 int max2(int a, int b){ return (a > b) ? a : b; } // ternary operator
2 int main(void){
3     int x = max2(3, 7);    // call: arguments copied into parameters
4     printf("%d\n", x);    // 7
5 }
```

C passes arguments *by value*: the function receives copies. To let a function modify the caller's variable, pass its address (pointer) — the subject of Ch. 2. Variables have *block scope*: visible only inside their `{}`.

1.6 Operators, Expressions, and Precedence

An *expression* combines values and operators to produce a new value. C evaluates using precedence (higher binds tighter) and associativity.

Prec.	Operator	Associativity
1	() [] -> .	left
2	! ~ ++ - + - * & (unary)	right
3	* / %	left
4	+ -	left
5	< <= > >=	left
6	== !=	left
7	&&	left
8		left
9	?:	right
10	= += -=	right

```

1 int a=5,b=2,c=3;
2 int r = a + b * c;    // 11, not 21: * before +
3 int s = (a+b)*c;      // 21: parentheses override
4 int t = a > b && b < c; // 1 (true): relational before logical
5 // Pitfall: == vs = (comparison vs assignment)
6 if(a == 5) printf("yes\n"); // correct
7 // if(a = 5) always true (assigns 5, yields 5 which is truthy) -- bug!
```

Integer division truncates towards zero: `7/2==3`, `7%2==1`. Mixing signed/unsigned promotes to unsigned — a frequent bug source. When in doubt, parenthesise.

1.7 Worked Example: Grading Program

We combine all ideas into a small program that reads scores and prints a grade, demonstrating I/O, branching, loops, and functions.

```

1  #include <stdio.h>
2  char grade_of(int score){
3      if(score>=80) return 'A';
4      if(score>=70) return 'B';
5      if(score>=50) return 'C';
6      return 'F';
7  }
8  int main(void){
9      int n; printf("How many scores? ");
10     if(scanf("%d",&n)!=1 || n<=0) return 1;
11     for(int i=0;i<n;i++){
12         int s; printf("Score %d: ", i+1);
13         scanf("%d",&s);
14         printf(" -> Grade %c\n", grade_of(s));
15     }
16     return 0;
17 }
```

Trace with input 2 85 62: $n=2$, loop $i=0$ reads 85 $\rightarrow$ A, $i=1$ reads 62 $\rightarrow$ C. Each function call gets its own frame (Ch. 2).

1.8 Chapter Notes

Kernighan & Ritchie *The C Programming Language* (2nd ed.) remains the canonical reference. `gcc -Wall -Wextra -std=c11` enables all warnings — treat warnings as errors early.

1.9 Exercises

- E1.1 **Trace.** What does this print? `int a=5,b=2; printf("%d %d %d",a/b, a%b, (double)a/b);` Explain each expression.
- E1.2 **scanf pitfalls.** Why does `scanf("%d", age)` (without `&`) often segfault? What does checking `scanf`'s return value protect against?
- E1.3 **Loop rewrite.** Rewrite `for(int i=0;i<n;i++) sum+=i;` as a `while` loop and as a `do-while` loop. When do the three forms behave differently?
- E1.4 **Fall-through bug.** Write a `switch` on `char grade` that maps `'A'/'B'/'C'/'D'/'F'` to messages, correctly using `break`. Show a version where a missing `break` causes a wrong message.
- E1.5 **Function scope.** Write `int factorial(int n)` iteratively and recursively. Call both from `main` and explain why the recursive version needs a base case; what happens without it?

Chapter 2

Pointers and Memory: Addresses as Data

“Pointers are the C programmer’s superpower and the beginner’s nightmare.” A pointer is just an address held in a variable. Master this one idea and C’s memory model becomes transparent.

From zero: we build on Ch. 1 boxes. Memory is a numbered array of bytes; a variable’s name hides its address. We reveal the address, store it, and draw every step. No prior systems knowledge assumed.

Learning Outcomes

After this chapter you will be able to:

- (L1) distinguish a variable’s value from its address (& vs. *);
- (L2) draw stack frames and explain call-by-value vs. call-by-reference;
- (L3) perform pointer arithmetic correctly for typed pointers;
- (L4) identify dangling, null, and wild pointers;
- (L5) explain the stack vs. heap distinction at a high level (detail in Ch. 4).

2.1 Addresses and the & Operator

Every byte in memory has an index — its *address* (like a house number). A variable `int x=42` occupies 4 bytes starting at some address, say `0x7ffc1000`. `&x` yields that address; `*p` follows a pointer to the box it points at.

```
1 int x = 42, y = 7;
2 int *p = &x;           // p holds address of x; type int* = pointer to int
3 printf("%p %d\n", (void*)p, *p); // 0x7ffc1000 42
4 *p = 100;               // writes through p: x is now 100
5 printf("x=%d\n", x);    // 100
6 int **pp = &p;          // pointer to pointer
```

`*p` is an *lvalue*: you can assign to it. `NULL` (i.e. `(void*)0`) means “points nowhere” — dereferencing it crashes.

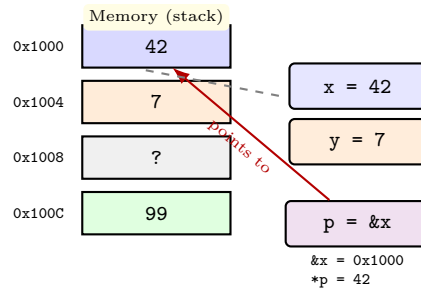


Figure 2.1: Memory as numbered boxes. `p` stores the address of `x`; dereferencing `*p` reads box `0x1000`.

2.2 The Stack: Frames on Call

Each function call pushes a *frame* (activation record) onto the call stack: parameters, local variables, return address.

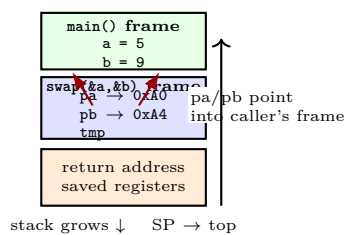


Figure 2.2: Stack frames: `main` calls `swap(&a, &b)`; the callee's pointers reach back into the caller's frame to modify `a, b`.

Call-by-value copies; to modify the caller, pass the address:

```

1 void swap(int *pa, int *pb){
2     int tmp = *pa; *pa = *pb; *pb = tmp;
3 }
4 int main(void){
5     int a=5, b=9; swap(&a, &b); // a=9 b=5
6 }

```

Without pointers, `swap(a, b)` would swap copies and the caller would see no change — a classic bug.

2.3 Pointer Arithmetic: Typed Steps

A pointer's type determines its stride. If `p` is `int*`, then `p+1` advances by `sizeof(int)` bytes (typically 4), not 1. The compiler scales for you.

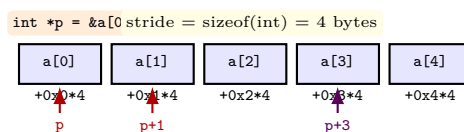


Figure 2.3: Typed pointer arithmetic: `p+1` jumps one `int` (4 bytes), not one byte. `a[i]` is sugar for `*(a+i)`.

```

1 int a[5]={10,20,30,40,50};
2 int *p = a; // array decays to &a[0]
3 printf("%d\n", *(p+2)); // 30 == a[2]

```

```

4 | printf("%d\n", p[3]);    // 40 == *(p+3)
5 | char *cp = (char*)p;    // cp+1 advances 1 byte
6 | printf("%ld %ld\n", (long)(p+1)-(long)p, (long)(cp+1)-(long)cp); // 4 1

```

`void*` has no stride (cannot do arithmetic); cast first. Subtracting two `int*` gives element distance, not byte distance.

2.4 Dangerous Pointers

`NULL` dereference, *wild* (uninitialised), *dangling* (points to freed/reclaimed stack frame), and *out-of-bounds* pointers are undefined behaviour. Always initialise: `int *q=NULL`; and check before dereferencing. Never return the address of a local variable.

2.5 Const Pointers and Pointer-to-Const

C distinguishes three related types; the position of `const` matters.

```

1 | int x=5, y=9;
2 | const int *p1 = &x;    // pointer to const int: *p1 cannot be changed, p1 can
3 | int *const p2 = &x;    // const pointer to int: p2 cannot be changed, *p2 can
4 | const int *const p3 = &x; // both const
5 | // p1 = &y; ok      *p1=10; error
6 | // p2 = &y; error *p2=10; ok

```

Read declarations right-to-left: “`p1` is a pointer to a `const int`”. Use `const int*` for read-only views of data you do not own.

2.6 Arrays, Pointers, and Function Parameters

An array name decays to a pointer in most contexts, but `sizeof` and `&` are exceptions (Ch. 3 elaborates). When you pass an array to a function, you really pass a pointer — the callee cannot know the length.

```

1 | void print_arr(const int *a, int n){
2 |     for(int i=0;i<n;i++) printf("%d ", a[i]); // a[i] == *(a+i)
3 |     // a is really int*: sizeof(a)==8 on 64-bit, not 20!
4 | }
5 | int main(void){
6 |     int arr[5]={1,2,3,4,5};
7 |     printf("sizeof arr=%zu\n", sizeof(arr)); // 20
8 |     print_arr(arr, 5);                      // arr decays to &arr[0]
9 | }

```

Always pass the length explicitly. For 2-D arrays, all dimensions except the first must be known: `void f(int a[][3], int rows)`.

2.7 Worked Example: Pass-by-Reference via Pointers

C has only pass-by-value. To let a function return two values, pass output pointers:

```

1 #include <stdbool.h>
2 bool divmod(int a, int b, int *q, int *r){
3     if(b==0) return false;
4     *q = a/b; *r = a%b; return true;
5 }
6 int main(void){
7     int q,r;
8     if(divmod(17,5,&q,&r)) printf("17/5 = %d rem %d\n", q,r); // 3 rem 2
9 }

```

The caller lends the address; the callee writes through it. This pattern — output parameters via pointers — pervades C APIs (`scanf` itself uses it).

2.8 Pointer Safety Checklist

Before dereferencing any pointer, ask four questions:

1. Is it initialised? (wild if not)
2. Is it NULL? (null dereference crashes)
3. Does it still point to live storage? (dangling if freed/returned local)
4. Is the access in bounds? (overflow corrupts neighbours)

Adopt the habit: initialise to NULL, check before use, null after `free`, never return locals. Tools like AddressSanitizer (`-fsanitize=address`) turn silent UB into immediate aborts with stack traces — use them during development.

```

1 int *p = NULL;           // 1. initialise
2 p = malloc(sizeof(*p));
3 if(!p) return;           // 2. check allocation
4 *p = 42;                  // 3. live and in bounds
5 free(p); p = NULL;        // 4. free + null (prevents dangling)
6 // if(p) *p = 1;         // safe: null check guards

```

2.9 Chapter Notes

The stack/heap split is elaborated in Ch. 4. For deeper C memory, see Bryant & O'Hallaron *CS:APP* Ch. 3, 9.

2.10 Exercises

- E2.1 **Trace.** `int x=3; int *p=&x; int y=*p+1; *p=10;` What are `x`, `y`, `*p` after each line? Draw the boxes.
- E2.2 **Swap failure.** Explain why `void bad_swap(int a,int b){int t=a;a=b;b=t;}` does not swap the caller's variables. Fix it with pointers.
- E2.3 **Arithmetic.** On a machine with 4-byte `int`, if `p=0x1000` is `int*`, what are `p+1`, `p+5` numerically? What about `(char*)p+1`?
- E2.4 **Dangling.** Why is `int *f(){int x=5; return &x;}` dangerous? What might go wrong when the caller uses the returned pointer?
- E2.5 **Double pointer.** Write `void set_to_zero(int **pp)` that allocates no memory but makes `*pp = NULL` visible to the caller. Why does `void foo(int *p){p=NULL;}` not do the same?

Chapter 3

Arrays, Strings, and Structs: Building Bigger Boxes

“Bad programmers worry about the code. Good programmers worry about data structures.” — Linus Torvalds. We now aggregate bytes into arrays, text into strings, and records into structs.

From zero: arrays are just many boxes in a row. We start with contiguous storage, show why indexing is arithmetic, then build strings (char arrays + terminator) and structs (heterogeneous boxes) with pictures before code.

Learning Outcomes

After this chapter you will be able to:

- (L1) declare, initialise, and index arrays and explain bounds checking (or lack thereof);
- (L2) manipulate C strings safely with `<string.h>` functions;
- (L3) define and use `struct` types with correct member access;
- (L4) relate arrays to pointers and explain decay and `sizeof` pitfalls;
- (L5) design a small record type and its helper functions.

3.1 Arrays: Contiguous Homogeneous Storage

An array packs N elements of the same type contiguously. The name `a` is the address of element 0; `a[i]` means `*(a+i)`.

```
1  int a[5] = {11,22,33,44,55};    // initialise
2  int b[5] = {0};                // all zero
3  // int c[5] = {1,2};           // remainder zero-filled
4  for(int i=0;i<5;i++) printf("%d ", a[i]);
5  printf("\nsize=%zu bytes\n", sizeof(a)); // 20 = 5*4
```

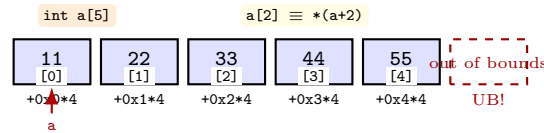



Figure 3.1: Array layout: 5 contiguous ints. Indexing is pointer arithmetic. Accessing beyond [4] is undefined behaviour (no bounds check in C).

```
6 // void f(int x[5]) -- inside f, sizeof(x) == sizeof(int*) !
```

C does *no* bounds checking. `a[5]` or `a[-1]` compiles but reads/writes arbitrary memory — a classic security hole. Multi-dimensional arrays are arrays of arrays: `int m[2][3]` is 6 ints row-major.

3.2 C Strings: Char Arrays with a Sentinel

A C string is a `char` array terminated by `'\0'` (value 0). The terminator marks the end; the array length may be larger.

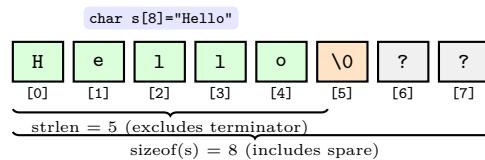


Figure 3.2: A C string "Hello" in an 8-byte buffer. `strlen` counts to `\0`; `sizeof` counts the whole array.

```
1 #include <string.h>
2 char s[16] = "Hello";           // 6 bytes used, 10 spare
3 printf("%zu %zu\n", strlen(s), sizeof(s)); // 5 16
4 strcpy(s, "Hi");                // copies including \0
5 strcat(s, "!");                 // appends: "Hi!"
6 // DANGER: strcpy with no bounds check -- use strncpy/strncpy or snprintf
7 snprintf(s, sizeof(s), "%s", "Hello, NTU!"); // safe, always terminates
8 for(int i=0; s[i]!='\0'; i++) putchar(s[i]);
```

String literals like "Hi" live in read-only memory; writing to them is UB. Always ensure a `\0` and enough space.

3.3 Structs: Heterogeneous Records

A `struct` groups different types into one compound type. Each *member* has its own offset.

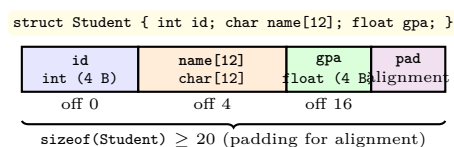


Figure 3.3: Struct layout: members placed in order with possible padding for alignment. `sizeof` includes padding.

```
1 struct Student{
```

```

2     int    id;
3     char   name[32];
4     float  gpa;
5 };
6 struct Student s = {123, "Asha", 4.8f};
7 printf("%s %.1f\n", s.name, s.gpa);
8 struct Student *ps = &s;
9 printf("%d\n", ps->id);           // -> for pointer to struct
10 // array of structs
11 struct Student cohort[50];
12 cohort[0].id = 1;

```

Access via `s.member` or `ps->member` (shorthand for `(*ps).member`). The compiler may insert padding between members for alignment; never assume `sizeof` equals sum of members. Use `typedef` to alias: `typedef struct Student Student;`

3.4 Arrays vs. Pointers — Decay

In most expressions, an array *decays* to a pointer to its first element. Exceptions: `sizeof(a)`, `&a`, and string literal initialisation. Hence function parameters declared as `int a[]` are really `int *a`; the callee cannot know the length — pass it explicitly: `void print(int *a, int n)`.

3.5 Searching and Sorting Arrays

With arrays we can implement linear search and bubble sort — the algorithmic core of SC1007, previewed here.

```

1 int linear_search(const int *a, int n, int key){
2     for(int i=0; i<n; i++) if(a[i]==key) return i;
3     return -1; // not found
4 }
5 void bubble_sort(int *a, int n){
6     for(int i=0; i<n-1; i++){
7         for(int j=0; j<n-1-i; j++){
8             if(a[j]>a[j+1]){ int t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
9         }
10    }

```

`linear_search` is $O(n)$; `bubble_sort` is $O(n^2)$. The sorted array can then be binary-searched in $O(\log n)$ — a theme of Ch. 7's `std::sort/find`.

3.6 Nested Structs and Typedef

Structs can contain other structs, enabling hierarchical records.

```

1 struct Date{ int day, month, year; };
2 struct Student{
3     int id;

```

```

4     char name[32];
5     float gpa;
6     struct Date enrolled;
7 };
8 typedef struct Student Student; // alias: Student instead of struct Student
9 Student s = {123, "Asha", 4.8f, {15, 8, 2024}};
10 printf("%02d/%02d/%04d\n", s.enrolled.day, s.enrolled.month, s.enrolled.year);
11 Student *ps = &s;
12 ps->gpa = 4.9f; // -> for pointer

```

`typedef` shortens declarations; modern C++ prefers using `Student = struct Student;`. Nested access chains like `s.enrolled.year` read left to right: “student’s enrolment date’s year”.

3.7 Common Pitfalls and Safe String Handling

C’s string functions are notoriously unsafe. Prefer bounded variants and always ensure termination.

```

1 char dst[8];
2 // UNSAFE: strcpy(dst, "Hello, NTU!"); // 12 bytes into 8 -- overflow!
3 snprintf(dst, sizeof(dst), "%s", "Hello, NTU!"); // safe: truncates to "Hello, "
4 printf("dst='%s' len=%zu\n", dst, strlen(dst)); // dst='Hello, ' len=7
5
6 char a[16]="Hello", b[]=" World";
7 // strncat with remaining space
8 strncat(a, b, sizeof(a)-strlen(a)-1); // leaves room for \0
9 printf("%s\n", a); // Hello World
10
11 // Tokenising (destructive)
12 char line[]="one,two,three";
13 for(char *tok=strtok(line,","); tok; tok=strtok(NULL,",")) printf("[%s]\n", tok);

```

Rule: every `char` buffer has a size; every string op must respect it. When in doubt, use `snprintf` and check its return value (needed length).

3.8 Chapter Notes

C strings are a frequent source of buffer overflows; modern code prefers bounded functions (`snprintf`, `strncpy`). C++ `std::string` (Ch. 7) eliminates the terminator bookkeeping entirely.

3.9 Exercises

E3.1 **sizeof trap.** `int a[10];` What are `sizeof(a)` and `sizeof(&a)`? Write a function `void foo(int a[10])` and explain why `sizeof(a)` inside `foo` differs.

E3.2 **String length.** Implement `int my_strlen(const char *s)` without using `<string.h>`. Why must the parameter be `const char*`?

- E3.3 **Off-by-one.** This copies unsafely: `char dst[8]; strcpy(dst, "Hello, NTU!");` How many bytes are written? Fix it with `snprintf` and explain the truncation behaviour.
- E3.4 **Struct padding.** Given `struct S{char c; int i; char d;};` predict `sizeof(struct S)` on a 4-byte-aligned machine. Verify with `printf("%zu", sizeof(struct S))`.
- E3.5 **Record design.** Define `struct Course{char code[8]; int au; float grade;}` and write `float gpa(struct Course *cs, int n)` computing the AU-weighted GPA.

Chapter 4

Dynamic Memory: The Heap and Its Discipline

“Memory management is the price of power.” Stack memory is automatic but fixed-size; heap memory is manual but flexible. This chapter teaches you to wield `malloc` and `free` safely.

From zero: stack vs. heap as two regions. The stack grows/shrinks automatically per call; the heap is a pool you request chunks from. We draw both, then build `malloc/free` discipline with `valgrind` as proof.

Learning Outcomes

After this chapter you will be able to:

- (L1) contrast stack and heap lifetime and size;
- (L2) allocate/free heap memory with `malloc/calloc/realloc/free`;
- (L3) diagnose leaks, double-free, and use-after-free with `valgrind/ASan`;
- (L4) implement a resizable array (vector) in C;
- (L5) state and apply the “every `malloc` has a `free`” discipline.

4.1 Stack vs. Heap

```
1 void foo(void){
2     int local = 5;           // stack: freed when foo returns
3     int *heap = malloc(sizeof(int)); // heap: lives until free(heap)
4     if(!heap) return;       // malloc can fail!
5     *heap = 5;
6     free(heap); heap = NULL; // release + null to avoid dangling
7 }
```

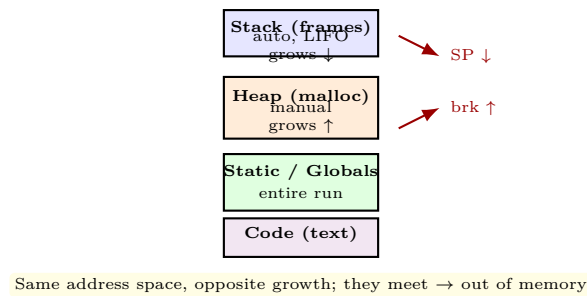


Figure 4.1: Process address space (simplified): stack grows down automatically; heap grows up on demand. Stack is per-call, heap is per-programmer-request.

Stack allocation is instant and reclaimed on return; heap allocation is explicit and survives the creating function — but you must free it.

4.2 malloc, calloc, realloc, free

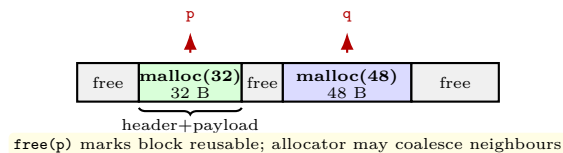


Figure 4.2: Heap as a sequence of blocks. `malloc` finds a free chunk, marks it allocated, returns its payload address. `free` marks it free again.

```

1  #include <stdlib.h>
2  #include <string.h>
3  int *a = malloc(10 * sizeof(*a));           // 10 ints, uninitialised
4  int *b = calloc(10, sizeof(*b));           // 10 ints, zeroed
5  if(!a || !b){ perror("malloc"); exit(1); }
6
7  a[0]=42; b[0]=42;                          // use
8  int *tmp = realloc(a, 20*sizeof(*a));      // grow to 20
9  if(tmp){ a=tmp; } else { /* handle failure, old a still valid */ }
10
11 free(a); free(b); a=b=NULL;                // always free what you malloc

```

Rules: check for `NULL`; `realloc` may move the block (use `tmp`); `free(NULL)` is safe no-op; never `free` stack addresses or the same pointer twice.

`sizeof(*a)` is preferred over `sizeof(int)` — it stays correct if `a`'s type changes.

4.3 Common Heap Bugs

- **Leak:** `malloc` without `free` — memory grows until exhaustion. Long-running programs crash.
- **Double free:** `free(p); free(p);` — corrupts allocator metadata.
- **Use-after-free:** `free(p); *p=1;` — writes to reclaimed memory; may corrupt next allocation.

- **Buffer overflow:** writing past the allocated size — corrupts adjacent block headers.

4.4 Valgrind and AddressSanitizer

Tools turn silent UB into loud errors. Compile with `-g` (debug symbols), then:

```

1 // leaky.c
2 int main(void){ malloc(100); return 0; } // leak!

gcc -g -Wall leaky.c -o leaky
valgrind --leak-check=full ./leaky          # reports 100 bytes lost
gcc -g -fsanitize=address leaky.c -o leaky_asan && ./leaky_asan # ASan report

```

Valgrind output names the allocation site and bytes lost; ASan aborts on the first illegal access with a stack trace. Run one of them on every heap-using program during development. On macOS, use `leaks` or ASan (valgrind is Linux-first).

4.5 A Resizable Array in C

We combine structs + heap to build a minimal vector:

```

1 typedef struct { int *data; int size; int cap; } Vec;
2 void vec_init(Vec *v){ v->data=NULL; v->size=0; v->cap=0; }
3 void vec_push(Vec *v, int x){
4     if(v->size==v->cap){
5         int nc = v->cap ? v->cap*2 : 4;
6         int *nd = realloc(v->data, nc*sizeof(*nd));
7         if(!nd){ perror("realloc"); exit(1); }
8         v->data=nd; v->cap=nc;
9     }
10    v->data[v->size++]=x;
11 }
12 void vec_free(Vec *v){ free(v->data); v->data=NULL; v->size=v->cap=0; }

```

Doubling amortises `realloc` cost to $O(1)$ per push. Every `vec_init` must be paired with `vec_free`.



Figure 4.3: Resizable array: heap buffer with spare capacity. Doubling keeps amortised $O(1)$ push and wastes at most half the buffer.

4.6 Deep Copy vs. Shallow Copy

When a struct owns heap memory, copying the struct copies the pointer, not the buffer — a shallow copy. Two structs then point to the same buffer; freeing both double-frees.

```

1 typedef struct { char *name; int age; } Person;
2 Person a; a.name = malloc(16); strcpy(a.name, "Asha"); a.age=20;
3 Person b = a; // shallow: b.name == a.name (same pointer!)
4 free(a.name); // b.name now dangling -- use-after-free if accessed
5 // Fix: deep copy
6 Person c; c.name = malloc(strlen(a.name)+1); strcpy(c.name, a.name); c.age=a.age;

```

In C++ (Ch. 5,8) copy constructors automate deep copying. In C you must do it manually, every time.

4.7 Worked Example: Dynamic String Builder

We build a string of unknown length by doubling, mirroring Vec but for chars.

```

1 #include <string.h>
2 typedef struct { char *buf; int len; int cap; } StrBuilder;
3 void sb_init(StrBuilder *sb){ sb->buf=malloc(16); sb->cap=16; sb->len=0; sb->buf[0]='\0';
4     }
5 void sb_append(StrBuilder *sb, const char *s){
6     int slen=strlen(s);
7     while(sb->len+slen+1 > sb->cap){
8         sb->cap*=2;
9         char *nb=realloc(sb->buf, sb->cap);
10        if(!nb){ perror("realloc"); exit(1); }
11        sb->buf=nb;
12    }
13    strcpy(sb->buf+sb->len, s); sb->len+=slen;
14 }
15 void sb_free(StrBuilder *sb){ free(sb->buf); sb->buf=NULL; sb->len=sb->cap=0; }

```

Every `sb_init` pairs with `sb_free`. The doubling keeps total copy cost $O(n)$ for n appended characters.

4.8 Chapter Notes

Heap allocators (first-fit, best-fit, segregated lists) are covered in SC2005 Operating Systems. For C++ alternatives that automate freeing, see Ch. 8 (RAII, smart pointers).

4.9 Exercises

E4.1 **Lifetime.** Why does returning `&local` from a function fail, but returning `malloc`'d memory succeeds? When must the caller `free` the latter?

E4.2 **calloc vs malloc.** When is `calloc(n, sizeof(T))` preferable to `malloc(n*sizeof(T))`? What does “zeroed” mean for pointer members?

E4.3 **realloc pitfall.** Explain why `a = realloc(a, new_size)` leaks on failure. Write the safe pattern with a temporary pointer.

- E4.4 **Valgrind reading.** Write a program that leaks 3 blocks and double-frees one. Run under valgrind/ASan and explain each diagnostic line.
- E4.5 **Vec shrink.** Extend `Vec` with `vec_pop` and a shrink policy (e.g. halve when `size == cap/4`). Argue why shrinking at `size == cap/2` could cause thrashing.

Chapter 5

C++ Classes and Object-Oriented Basics

“C++ is C with a better abstraction toolkit.” A `class` bundles data with the operations that maintain its invariants — turning a loose struct into a disciplined object.

From zero: C structs $\rightarrow$ C++ classes. We start with why bare structs are error-prone (anyone can break invariants), then add constructors, encapsulation, and methods step by step. No prior OOP assumed.

Learning Outcomes

After this chapter you will be able to:

- (L1) define a C++ class with access control (`private/public`);
- (L2) write constructors, destructors, and use member-initialiser lists;
- (L3) distinguish value vs. reference semantics and use `const` correctly;
- (L4) explain encapsulation and class invariants;
- (L5) trace object lifetime (construction $\rightarrow$ use $\rightarrow$ destruction).

5.1 From Struct to Class

A C struct is passive: any code can set `s.gpa = 999`. A class makes data `private` and exposes only safe operations.

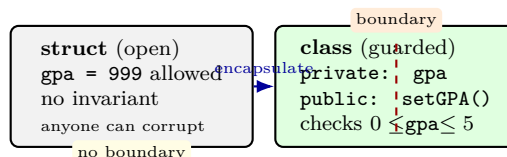


Figure 5.1: Encapsulation: a class erects a wall. Outside code must use the public interface, which can enforce invariants.

```

1 #include <string>
2 class Student{
3 private:
4     int id_;
5     std::string name_;
6     double gpa_;           // invariant: 0.0 <= gpa_ <= 5.0
7 public:
8     Student(int id, std::string name, double gpa) // constructor
9         : id_(id), name_(std::move(name)), gpa_(gpa) {
10         if(gpa_<0 || gpa_>5) throw std::out_of_range("gpa");
11     }
12     double gpa() const { return gpa_; }           // getter, const = won't mutate
13     void setGPA(double g){                         // setter with check
14         if(g<0||g>5) throw std::out_of_range("gpa");
15         gpa_=g;
16     }
17     std::string name() const { return name_; }
18 };

```

class defaults to private; struct defaults to public — otherwise identical in C++.

5.2 Constructors, Destructors, Initialiser Lists

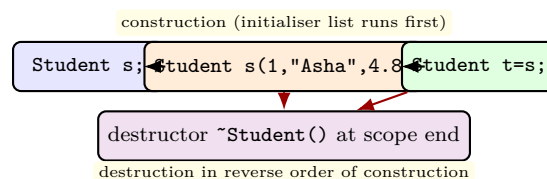


Figure 5.2: Object lifetime: construction via initialiser list, then use, then automatic destruction when the scope ends. Copy construction is discussed in Ch. 8.

```

1 class Vec2{
2     double x_, y_;
3 public:
4     Vec2(): x_(0), y_(0) {}           // default constructor
5     Vec2(double x, double y): x_(x), y_(y) {} // parameterised
6     // member initialiser list : x_(x) runs BEFORE body -- required for const/ref members
7     double norm() const { return std::sqrt(x_*x_+y_*y_); }
8     ~Vec2(){ /* release resources if any */ } // destructor
9 };
10 Vec2 a;           // default
11 Vec2 b(3,4);      // param
12 Vec2 c = b;       // copy (memberwise for now; Ch.8 refines)

```

Prefer initialiser lists over assignment in the body: they directly construct members, while assignment constructs then overwrites.

5.3 Encapsulation and Invariants

A class invariant is a predicate that every public operation must preserve (e.g. $0 \leq \text{gpa} \leq 5$, vector `size` $\leq$ `capacity`). By making data private and checking in setters/constructors, the invariant holds for all clients. Think of the class as a tiny machine with a valid-state set; public methods are the only buttons.

```

1  class BoundedCounter{
2      int val_, max_;
3  public:
4      BoundedCounter(int m): val_(0), max_(m) {}
5      void inc(){ if(val_ < max_) ++val_; }
6      void reset(){ val_=0; }
7      int get() const { return val_; }
8      // invariant: 0 <= val_ <= max_ (always, between calls)
9  };

```

5.4 Const Correctness and References

```

1  void print(const Student& s){           // const ref: no copy, no mutate
2      std::cout << s.name() << " " << s.gpa() << "\n";
3  }
4  Student s(1, "Asha", 4.8);
5  print(s);                             // binds without copying string
6  // s.gpa() is const member function -- callable on const Student&

```

`const` on a method means `this` is `const`; it can be called on `const` objects. Pass large objects by `const&` to avoid copies. Return by value for small types, by `const` ref for members (with lifetime care).

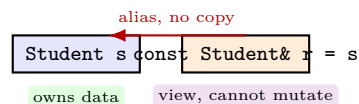


Figure 5.3: Reference semantics: `const Student&` is an alias (no copy) with read-only access. Value semantics would copy the whole object.

5.5 Operator Overloading

Classes can give meaning to operators like `+`, `==`, `<<` so objects feel like built-in types.

```

1  class Vec2{
2      double x_, y_;
3  public:
4      Vec2(double x=0, double y=0): x_(x), y_(y) {}
5      Vec2 operator+(const Vec2& o) const { return {x_+o.x_, y_+o.y_}; }
6      bool operator==(const Vec2& o) const { return x_==o.x_ && y_==o.y_; }
7      double x() const { return x_; }
8      double y() const { return y_; }
9      friend std::ostream& operator<<(std::ostream& os, const Vec2& v){
10         return os<<" ("<<v.x_<<" "<<v.y_<<") ";

```

```

11     }
12 };
13 Vec2 a(1,2), b(3,4);
14 Vec2 c = a+b;           // (4,6)
15 std::cout<<c<<"\n";    // (4,6)

```

Overload only when semantics are obvious; `+` should not surprise. Stream insertion `operator<<` is idiomatically a `friend` non-member.

5.6 Static Members and Friends

`static` members belong to the class, not to any instance; `friend` grants selective access to private data.

```

1 class Counter{
2     static int total_;    // shared across all objects
3     int id_;
4 public:
5     Counter(): id_(++total_) {}
6     static int total(){ return total_; }
7     friend void debug(const Counter& c); // can read c.id_ private
8 };
9 int Counter::total_ = 0; // define static outside class
10 void debug(const Counter& c){ std::cout<<c.id_<<"\n"; }

```

Static data needs an out-of-class definition (or `inline static` since C++17). Friends break encapsulation minimally — prefer member functions when possible.

5.7 Member Functions vs. Non-Members

Not every operation should be a member. Prefer non-member non-friend when it can be implemented via the public interface — it reduces coupling and keeps the class minimal.

```

1 class Vec2{
2     double x_, y_;
3 public:
4     Vec2(double x=0, double y=0): x_(x), y_(y) {}
5     double x() const { return x_; }
6     double y() const { return y_; }
7     Vec2& operator+=(const Vec2& o){ x_+=o.x_; y_+=o.y_; return *this; }
8 };
9 // non-member built from public members + +=
10 Vec2 operator+(Vec2 a, const Vec2& b){ a+=b; return a; }
11 double dot(const Vec2& a, const Vec2& b){ return a.x()*b.x()+a.y()*b.y(); }
12 bool isZero(const Vec2& v){ return v.x()==0 && v.y()==0; }

```

If a function needs private access, make it a member or friend; otherwise keep it outside. This is the “minimal interface” principle (Meyers).

5.8 Chapter Notes

Stroustrup *The C++ Programming Language* (4th ed.) and Meyers *Effective C++* are the classic guides. Rule of Zero/Three/Five (when to write copy/move/destructor) is covered in Ch. 8.

5.9 Exercises

- E5.1 **Invariant.** Design `class Date{int d,m,y}` with invariant “valid calendar date”. Which operations must validate? Sketch constructor and `setDay`.
- E5.2 **Initialiser list.** Why must a `const int id_` or `int& ref_` member be initialised in the member-initialiser list, not in the body?
- E5.3 **Const.** Mark which methods should be `const` in a `BankAccount` class (`balance()`, `deposit()`, `withdraw()`, `getId()`). What breaks if you omit `const`?
- E5.4 **Encapsulation break.** Show how returning a non-const reference to a private member (`int& getVal()`) breaks encapsulation. Fix it.
- E5.5 **Lifetime.** Trace construction/destruction order for `{ Student a(1,"A",4); Student b=a; }` How many constructor/destructor calls? Which kind?

Chapter 6

Inheritance, Polymorphism, and Templates

“Inheritance is sharing; polymorphism is substituting; templates are stamping.” Three ways to reuse code, each with a distinct cost model.

From zero: reuse without copy-paste. We start with why duplication hurts, then derive is-a hierarchies, virtual dispatch, and generic stamping from first principles.

Learning Outcomes

After this chapter you will be able to:

- (L1) model is-a relationships with public inheritance;
- (L2) explain virtual dispatch and the vtable mechanism;
- (L3) use templates for generic functions and classes;
- (L4) distinguish static (compile-time) vs. dynamic (run-time) polymorphism;
- (L5) apply access specifiers and virtual destructors correctly.

6.1 Inheritance: Is-A

If `Dog` is-a `Animal`, every `Dog` has the `Animal` part plus its own extensions. Inheritance captures this.

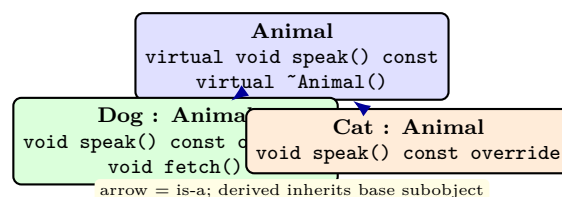


Figure 6.1: Inheritance hierarchy: `Dog` and `Cat` each contain an `Animal` subobject. Public inheritance models is-a.

```
1 struct Animal{
```

```

2   virtual void speak() const { std::cout<<"...\n"; }
3   virtual ~Animal() = default;           // virtual dtor: essential!
4 };
5 struct Dog : public Animal{
6     void speak() const override { std::cout<<"Woof!\n"; }
7     void fetch() const { std::cout<<"*fetches*\n"; }
8 };
9 Dog d; Animal& a = d; // Dog is-an Animal: binds base ref to derived
10 a.speak();           // Woof! (virtual dispatch)

```

`override` makes the compiler check the signature matches; always use it. Make base destructors `virtual` so `delete` through base pointer destroys the derived part.

6.2 Polymorphism and Virtual Dispatch

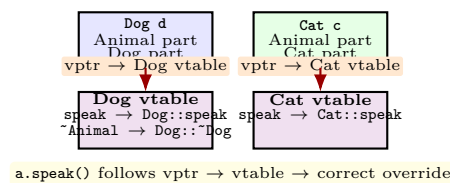


Figure 6.2: Virtual dispatch: each object holds a hidden `vptra` to its class's vtable. A virtual call indirections through it, selecting the derived override at run time.

```

1 void chorus(const std::vector<Animal*>& v){
2     for(auto *a: v) a->speak(); // dynamic dispatch: Dog woofs, Cat meows
3 }
4 Dog d; Cat c;
5 chorus({&d,&c}); // Woof! Meow!

```

Cost: one extra indirection per virtual call. Non-virtual calls are direct (zero overhead). Prefer non-virtual by default; make virtual only when you need run-time substitution. Pure virtual (`=0`) makes a base abstract: `virtual void draw() const =0;`

6.3 Templates: Compile-Time Stamping

Templates generate code per type at compile time (static polymorphism). No vtable, no indirection — but one instantiation per distinct type.

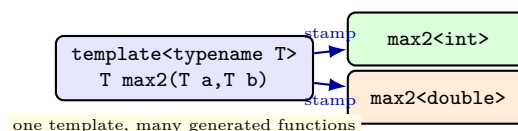


Figure 6.3: Templates as stamps: the compiler generates a separate function/class per `T`. No run-time cost, but code bloat if overused.

```

1 template<typename T>
2 T max2(T a, T b){ return (a>b)?a:b; }
3 int x = max2<int>(3,7);           // explicit

```



```

4 double y = max2(3.1, 2.9);           // deduced: T=double
5 // Class template
6 template<typename T>
7 struct VecT{ T x,y; T dot(const VecT& o) const { return x*o.x + y*o.y; } };
8 VecT<double> v{1.0,2.0};

```

Templates are type-checked after instantiation; errors can be verbose. C++20 concepts constrain T with readable diagnostics (preview in Ch. 8).

6.4 Static vs. Dynamic Polymorphism

	Templates (static)	Virtual (dynamic)
When chosen	compile time	run time
Cost	code size per T	one vtable indirection
Heterogeneous container	no (one T per container)	yes (<code>vector<Animal*></code>)

Rule of thumb: use templates for generic algorithms on uniform types; use virtual for heterogeneous runtime substitution.

6.5 Access Control and Protected Members

public inheritance keeps is-a; private/protected inheritance is implementation reuse, not is-a. protected members are visible to derived classes but not to outsiders.

```

1 class Base{
2 protected: int prot_ = 42; // derived can see, outsiders cannot
3 private:   int priv_ = 0;
4 public:    int pub_ = 1;
5 };
6 class Derived : public Base{
7 public:
8     void foo(){ prot_ = 7; /* ok */ /* priv_=1; error */ }
9 };
10 Base b; // b.prot_ error -- protected not accessible outside

```

Prefer private data with protected accessors over protected data: it preserves the base's invariant while still allowing customisation.

6.6 Abstract Classes and Interfaces

A class with at least one pure virtual function (=0) is abstract — it cannot be instantiated, only derived from. This models interfaces.

```

1 struct Drawable{ // interface: pure virtual
2     virtual void draw() const = 0;

```

```

3     virtual ~Drawable() = default;
4 };
5 struct Circle : Drawable{
6     double r; explicit Circle(double r_): r(r_) {}
7     void draw() const override { std::cout<<"Circle r="<<r<<"\n"; }
8 };
9 struct Square : Drawable{
10    double s; explicit Square(double s_): s(s_) {}
11    void draw() const override { std::cout<<"Square s="<<s<<"\n"; }
12 };
13 void render(const std::vector<Drawable*>& v){
14     for(auto *d: v) d->draw(); // uniform interface
15 }

```

Interfaces decouple callers from concrete types — the foundation of plugin architectures and test mocks. In modern C++, pure abstract classes are preferred over deep hierarchies.

6.7 Template Specialisation and Non-Type Parameters

Templates can be specialised for specific types or take compile-time constants as parameters.

```

1 template<typename T, int N> // non-type param: array size known at compile time
2 struct Array{
3     T data[N];
4     T& operator[](int i){ return data[i]; }
5     constexpr int size() const { return N; }
6 };
7 Array<int,5> a; // 5 ints, no heap, size() is compile-time
8
9 // Function template with overload for const char*
10 template<typename T> void print2(T x){ std::cout<<x<<"\n"; }
11 template<> void print2<const char*>(const char* s){ std::cout<<"C-string: "<<s<<"\n"; }
12 print2(42); // int version
13 print2("hello"); // specialised for const char*

```

Non-type parameters enable fixed-size containers without dynamic allocation; specialisation lets you tailor behaviour per type while keeping a uniform call site.

6.8 Chapter Notes

Sutter & Alexandrescu *C++ Coding Standards* Ch. 37–40 covers inheritance discipline. STL (Ch. 7) is almost entirely templates; inheritance appears sparingly there by design.

6.9 Exercises

- E6.1 **Slicing.** `Animal a = Dog(); a.speak();` What prints and why? How does `Animal& a = d;` differ? Explain object slicing.
- E6.2 **Virtual dtor.** Give a program where deleting a `Dog` through `Animal*` without a virtual destructor leaks resources. Fix it.
- E6.3 **Template deduction.** For `template<typename T> void f(T a, T b)`, what happens with `f(1, 2.0)`? How to fix the call or the template?
- E6.4 **Override check.** Remove `override` and misspell `speak` as `speal` in `Dog`. What happens with and without `override`? Why is `override` best practice?
- E6.5 **Design.** Model `Shape` with `area()` and `draw()`. Should they be virtual/pure virtual? Should `Shape` be instantiable? Justify and sketch the hierarchy.

Chapter 7

The Standard Template Library: Vectors, Maps, and Iterators

“STL is not a library you call — it is a language you think in.” Containers hold data; iterators traverse it; algorithms operate on it. Learn the trinity and you read modern C++ fluently.

From zero: we generalise Ch. 4’s Vec and Ch. 3’s arrays. STL containers automate the heap discipline; iterators generalise pointers; algorithms generalise loops. Pictures before code throughout.

Learning Outcomes

After this chapter you will be able to:

- (L1) use `vector`, `string`, `map/unordered_map`, `set` idiomatically;
- (L2) traverse any container with iterators and `range-for`;
- (L3) apply STL algorithms (`sort`, `find`, `accumulate`);
- (L4) state complexity guarantees of key containers;
- (L5) choose the right container for a task.

7.1 Sequence Containers: `vector` and `string`

`std::vector<T>` is Ch. 4’s resizable array done right: heap buffer, size, capacity, automatic growth, RAII cleanup.

```
1 #include <vector>
2 #include <string>
3 std::vector<int> v = {10,20,30};
4 v.push_back(40);           // may reallocate; amortised O(1)
5 v[0]=99; v.at(1)=21;       // at() bounds-checks, [] does not
6 for(int x: v) std::cout<<x<<" "; // range-for (sugar for iterators)
```

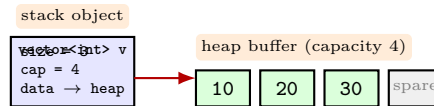


Figure 7.1: vector layout: small stack handle (size/cap/pointer) owns a heap array. Growth doubles capacity; spare slots avoid reallocation on push.

```

7  std::string s="Hello";
8  s += " NTU";           // string is vector<char> with extras
9  s.substr(0,5);         // "Hello"

```

Reserve when you know the size: `v.reserve(1000)` avoids repeated reallocations. References/pointers into a vector may dangle after reallocation — a common bug.

7.2 Associative Containers: map, set, unordered_map

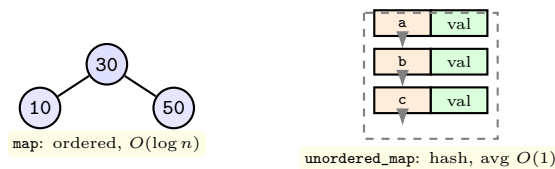


Figure 7.2: Ordered map (balanced BST, sorted) vs. unordered_map (hash table, unsorted but faster average).

```

1  #include <map>
2  #include <unordered_map>
3  #include <set>
4  std::map<std::string,int> scores;           // ordered by key
5  scores["Asha"]=95; scores["Bo"]=82;
6  for(auto &kv: scores) std::cout<<kv.first<<":"<<kv.second<<"\n"; // sorted
7
8  std::unordered_map<std::string,int> h = {"Asha",95};
9  h["Bo"]=82;                               // avg O(1)
10 if(h.find("Asha")!=h.end()) std::cout<<"found\n";
11 std::set<int> s={3,1,4,1,5};               // {1,3,4,5} sorted unique

```

Complexity: map/set $O(\log n)$ per op (red-black tree); unordered_map/unordered_set average $O(1)$, worst $O(n)$ on hash collision. Prefer unordered when order is irrelevant.

7.3 Iterators: Generalised Pointers

An iterator is a pointer-like object that knows how to step through a container. All STL algorithms speak iterators, not containers.

```

1  #include <algorithm>
2  #include <numeric>
3  std::vector<int> v={4,1,3,2};
4  std::sort(v.begin(), v.end());           // 1 2 3 4
5  auto it = std::find(v.begin(), v.end(), 3); // iterator to 3
6  if(it!=v.end()) std::cout<<"at " <<(it-v.begin())<<"\n";

```

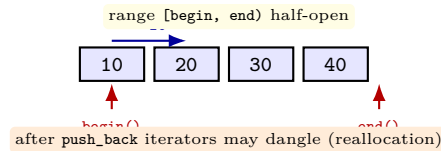


Figure 7.3: Iterator range `[begin, end)`: `begin` points to first element, `end` is one past the last. Increment advances; dereference reads.

```

7  int sum = std::accumulate(v.begin(), v.end(), 0); // 10
8  // range-for is sugar:
9  // for(auto it=v.begin(); it!=v.end(); ++it) ...
10 for(int &x: v) x*=2; // mutate via ref

```

Iterator categories: random-access (vector), bidirectional (map), forward (forward_list). Algorithms state which they require.

7.4 Choosing a Container

Need	Use	Why
Resizable array	<code>vector</code>	cache-friendly, $O(1)$ push
Sorted lookup	<code>map/set</code>	$O(\log n)$, ordered
Fast lookup, no order	<code>unordered_map</code>	avg $O(1)$
Unique sorted values	<code>set</code>	dedup + order

7.5 STL Algorithms in Depth

Algorithms are function templates that operate on iterator ranges. They never modify container structure directly; they read/write through iterators.

```

1  #include <algorithm>
2  #include <numeric>
3  std::vector<int> v={5,2,8,1,9};
4  // sort, reverse, count, transform
5  std::sort(v.begin(), v.end()); // 1 2 5 8 9
6  std::reverse(v.begin(), v.end()); // 9 8 5 2 1
7  int cnt = std::count(v.begin(), v.end(), 5); // 1
8  std::vector<int> w(v.size());
9  std::transform(v.begin(), v.end(), w.begin(),
10                [](int x){ return x*2; }); // w = 18 16 10 4 2
11 int sum = std::accumulate(v.begin(), v.end(), 0); // 25
12 bool any = std::any_of(v.begin(), v.end(),
13                        [](int x){ return x>8; }); // true

```

All algorithms work on any container with the required iterator category. Prefer algorithms over hand-written loops: they are optimised, self-documenting, and less error-prone.

7.6 Complexity Guarantees and Big-O of Containers

Operation	<code>vector</code>	<code>map</code>	<code>unordered_map</code>
<code>push_back</code>	amort. $O(1)$	—	—
<code>find</code>	$O(n)$	$O(\log n)$	avg $O(1)$
<code>insert</code>	$O(n)$ (middle)	$O(\log n)$	avg $O(1)$
<code>operator[]</code>	$O(1)$	$O(\log n)$	avg $O(1)$
Memory	contiguous	tree nodes	hash buckets

These guarantees drive container choice. `vector` wins for iteration (cache locality); `map` wins when order matters; `unordered_map` wins for raw lookup speed.

7.7 Chapter Notes

Josuttis *The C++ Standard Library* is the STL handbook. Complexity details are in the standard and at cppreference.com. Do not store raw owning pointers in containers; use values or smart pointers (Ch. 8).

7.8 Exercises

- E7.1 **Invalidation.** `auto it=v.begin(); v.push_back(99); *it=1;` When can this be UB? How does `reserve` help? What about `std::list`?
- E7.2 **Map vs unordered.** You need to count word frequencies and later print them alphabetically. Which map type? What if you only need fast lookup?
- E7.3 **Algorithm.** Use `std::sort + std::find` to sort a vector and locate a value. What iterator does `find` return on failure? How to check?
- E7.4 **Complexity.** State big- O for `vector::push_back`, `map::find`, `unordered_map::find` (average/worst). When does unordered degrade?
- E7.5 **Range-for desugar.** Rewrite `for(auto &x: v) x++` using explicit iterators. Explain why `auto &` vs. `auto` matters.

Chapter 8

Modern C++: RAII, Smart Pointers, and Move Semantics

“Modern C++ is not C with extras — it is a different language that happens to compile C.” RAII makes resource safety automatic; smart pointers make ownership explicit; move semantics makes copies cheap.

From zero: Ch. 4’s discipline was manual; now we automate it. Every resource gets an owner object whose destructor frees it. We build RAII, then smart pointers, then moves — each fixing a failure mode of the last.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain RAII and why it prevents leaks even on exceptions;
- (L2) use `unique_ptr`, `shared_ptr`, `weak_ptr` correctly;
- (L3) distinguish lvalues from rvalues and apply move semantics;
- (L4) state the Rule of Zero/Three/Five;
- (L5) write exception-safe code with RAII types.

8.1 RAII: Resource Acquisition Is Initialisation

Idea: tie every resource (memory, file, lock) to an object’s lifetime. Constructor acquires; destructor releases. No manual `free/close` — the scope does it, even if an exception is thrown.

```
1 #include <fstream>
2 void process(){
3     std::ofstream out("log.txt"); // acquires file handle
```

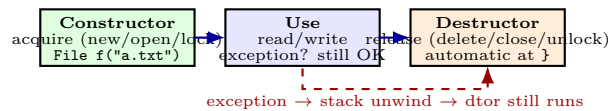



Figure 8.1: RAII timeline: acquisition in constructor, automatic release in destructor. Stack unwinding guarantees the destructor runs even on exception.

```

4   out << "hello\n";
5   if(some_error()) throw std::runtime_error("oops");
6   // no close needed -- destructor closes even on throw
7 } // out.~ofstream() runs here
  
```

`std::string`, `std::vector`, `std::lock_guard`, `std::fstream` are all RAII types. Manual `new/delete` is the non-RAII alternative — avoid it.

8.2 Smart Pointers: Explicit Ownership

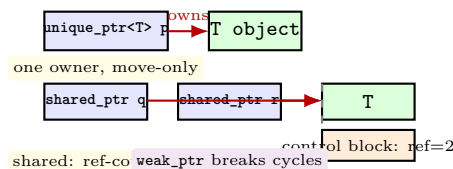


Figure 8.2: Smart pointers: `unique_ptr` has one owner; `shared_ptr` shares via a control block with reference count; `weak_ptr` observes without owning.

```

1  #include <memory>
2  auto p = std::make_unique<int>(42);           // unique ownership
3  // auto q = p;                               // error: not copyable
4  auto q = std::move(p);                        // transfer ownership; p==nullptr
5
6  auto a = std::make_shared<std::string>("hello"); // shared, ref count 1
7  auto b = a;                                   // ref count 2, same object
8  std::cout << a.use_count() << "\n";           // 2
9  std::weak_ptr<std::string> w = a;              // weak: doesn't increment count
10 // w.lock() -> shared_ptr if object still alive
  
```

Prefer `make_unique`/`make_shared` over raw `new`. Use `unique_ptr` by default; `shared_ptr` only when shared ownership is genuinely needed. `weak_ptr` breaks cycles (A owns B, B observes A weakly).

8.3 Move Semantics: Stealing Instead of Copying

Copying a vector duplicates its heap buffer ($O(n)$). *Moving* steals the buffer pointer and nulls the source ($O(1)$). Moves fire on *rvalues* (temporaries, `std::move`'d values).

```

1  std::vector<int> a = {1,2,3,4,5};
2  std::vector<int> b = a;                // copy: O(n)
3  std::vector<int> c = std::move(a);     // move: O(1), a is now empty
4  // a is valid but unspecified -- only safe to assign/destroy
  
```

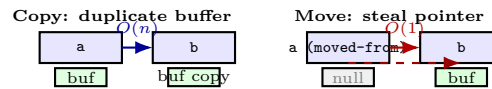


Figure 8.3: Copy duplicates the heap buffer; move transfers the pointer, leaving the source empty. Move is $O(1)$ vs. $O(n)$.

```
5 std::vector<int> make_vec(){ return {1,2,3}; } // NRVO/move elision
6 auto d = make_vec();                      // no copy (elided)
```

The compiler auto-moves temporaries; write `std::move` only for named lvalues you no longer need. After move, treat the source as empty.

8.4 Rule of Zero / Three / Five

- **Rule of Zero:** if your class uses RAII members (`string`, `vector`, smart pointers), write *no* custom destructor/copy/move — the compiler does it correctly.
- **Rule of Five:** if you *must* write one of destructor, copy-ctor, copy-assign, move-ctor, move-assign, you likely need all five.
- Prefer Zero; fall to Five only for low-level resource wrappers.

```
1 // Rule of Zero: no custom special members needed
2 class Student{
3     std::string name_; std::vector<int> scores_;
4 public:
5     Student(std::string n): name_(std::move(n)) {}
6     // compiler-generated ctor/copy/move are correct
7 };
```

8.5 Exception Safety and RAII in Practice

RAII makes exception safety almost free. Consider manual vs. RAII resource handling when an exception is thrown:

```
1 void manual(){
2     int *p = new int[100];
3     FILE *f = fopen("data.txt","r");
4     risky_operation(); // if throws, leak p and f!
5     delete[] p; fclose(f);
6 }
7 void raii(){
8     auto p = std::make_unique<int[]>(100);
9     std::ifstream f("data.txt"); // RAII file handle
10    risky_operation(); // if throws, destructors still run -- no leak
11 } // p and f destroyed automatically
```

Exception safety levels: *basic* (no leak), *strong* (rollback on failure), *nothrow* (never throws). RAII types give basic safety by default; transactions give strong safety.

8.6 Perfect Forwarding and Modern Idioms

C++11 introduced `auto`, range-for, lambdas, and forwarding references. C++17/20 added structured bindings and concepts.

```

1  auto add = [](auto a, auto b){ return a+b; }; // generic lambda (C++14)
2  std::cout<<add(3,4)<<" "<<add(3.1,2.9)<<"\n"; // 7 6.0
3
4  std::map<std::string,int> m={{ "Asha",95},{ "Bo",82}};
5  for(auto &[name,score]: m) // structured binding (C++17)
6      std::cout<<name<<": "<<score<<"\n";
7
8  // Perfect forwarding: preserve lvalue/rvalue nature
9  template<typename T>
10 void wrapper(T&& arg){ // forwarding reference
11     callee(std::forward<T>(arg)); // forwards as l- or r-value
12 }
```

These idioms reduce boilerplate and let the compiler generate optimal code. Prefer `auto` when the type is obvious; prefer range-for over index loops; prefer lambdas over hand-written functors.

8.7 Chapter Notes

Meyers *Effective Modern C++* (2014) is the definitive move/RAII guide. C++ Core Guidelines: “R.1: Manage resources automatically using RAII.”

8.8 Exercises

- E8.1 **RAII proof.** Write a function that opens two files and throws midway. Show RAII handles close it correctly while manual `fopen/fclose` leaks the second file.
- E8.2 **Ownership.** When would you choose `unique_ptr` vs. `shared_ptr` vs. raw `T*` (non-owning)? Give an example of each.
- E8.3 **Move trace.** `vector<int> a={1,2}; vector<int> b=std::move(a); cout<<a.size()<<" "<<b.size();` What prints? Is `a[0]` safe to read after the move?
- E8.4 **Rule of Zero.** A class holds `string name_` and `vector<double> grades_`. Do you need a custom destructor? What if it held `int* buf` from `new int[10]`?
- E8.5 **Cycle.** `struct A{shared_ptr<B> b;}; struct B{shared_ptr<A> a;};` Why does this leak? Fix it with `weak_ptr` and explain the ref counts.